Rahil Badkul
and Qile Jiang

Horn-Schunck Method

Introduction

Goal: Estimate pixel motion between two consecutive frames in a video.

Horn–Schunck method: Minimize the energy functional

$$E = \iint \underbrace{(I_x u + I_y v + I_t)^2}_{\text{constant brightness}} + \underbrace{\alpha^2(\|\nabla u\|^2 + \|\nabla v\|^2)}_{\text{spatial smoothness}} \, dx \, dy$$

where:

- ***Ix, Iy, It*** : derivatives of image intensity ***I*** along the x, y, and t.
- α : a regularization constant.

Introduction

The functional ***E*** can be minimized by solving the **Euler–Lagrange** equations:

$$\frac{\partial L}{\partial u} - \frac{\partial}{\partial x}\frac{\partial L}{\partial u_x} - \frac{\partial}{\partial y}\frac{\partial L}{\partial u_y} = 0$$
$$\frac{\partial L}{\partial v} - \frac{\partial}{\partial x}\frac{\partial L}{\partial v_x} - \frac{\partial}{\partial y}\frac{\partial L}{\partial v_y} = 0$$

$$I_x(I_x u + I_y v + I_t) - \alpha^2 \Delta u = 0$$
$$I_y(I_x u + I_y v + I_t) - \alpha^2 \Delta v = 0$$

where ***L*** is the integrand of ***E***, and ***Δ*** is the Laplace operator.

Using finite difference approximation, we get an **iterative scheme**:

$$u^{k+1} = \overline{u}^k - \frac{I_x(I_x\overline{u}^k + I_y\overline{v}^k + I_t)}{\alpha^2 + I_x^2 + I_y^2}$$

$$v^{k+1} = \overline{v}^k - \frac{I_y(I_x\overline{u}^k + I_y\overline{v}^k + I_t)}{\alpha^2 + I_x^2 + I_y^2}$$

where $\overline{u}(x,y)$ is a weighted avg. of u around the pixel at (x,y):

1/12	1/6	1/12
1/6	*	1/6
1/12	1/6	1/12

Implement H-S on GPU

Input: Frame pair **(I_1, I_2)**

Step 1: Compute Intensity Derivatives (GPU kernel)
- Apply Sobel derivative operator → **Ix, Iy**
- Temporal difference → **It**
- Obtain values for each pair of frames

$$G_x = \frac{1}{8}\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

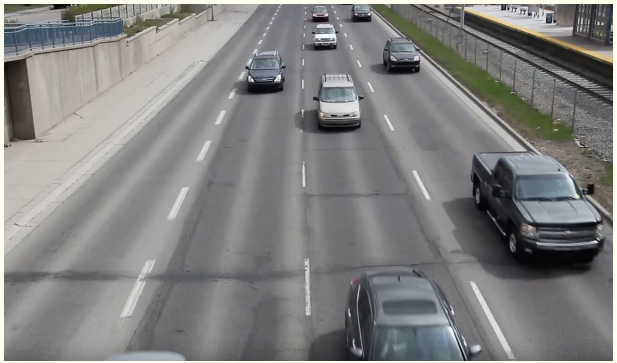
$$G_y = \frac{1}{8}\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Step 2: Iterative Solver (GPU kernel)
- Initialize: U = 0, V = 0
- Loop num_iter: Update **U, V** from neighbors, Swap **(U_old ↔ U_new)**

Output: Flow field **(U, V)**

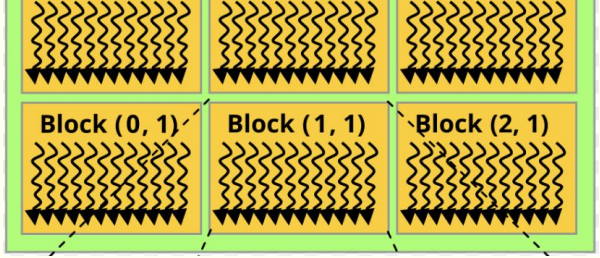
Implement H-S on GPU

One frame



height = 1080 pixels

width = 1920 pixels

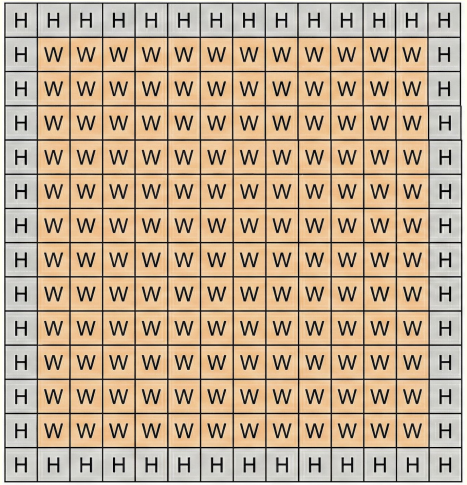



```
// Each block uses 18×18 threads to process a
// 16×16 pixel region + Halo around the region
dim3 block(18, 18);

// Divide a frame into a grid of blocks
dim3 grid((width + 15) / 16, (height + 15) / 16);
```

H-S with Shared Memory


```
__global__ void compute_derivatives(...){

  // Step 1: Shared mem. declaration: 16x16 block + 1-pixel halo
  __shared__ float s_tile[18][18];
  int x = bx * 16 + tx - 1;   // find global x coordinate
  int y = by * 16 + ty - 1;   // find global y coordinate

  // Step 2: Load data into shared memory
  s_tile[ty][tx] = I1[y * width + x];
   __syncthreads();       // barrier to make sure all threads finish

  // Step 3: only compute derivatives on internal pixels
  if (tx >= 1 && tx <= 16 && ty >= 1 && ty <= 16) {
    float I_ul   = s_tile[ty - 1][tx - 1];   // read from shared mem.
    float I_um = ...

    float gx = ...        // compute horizontal sobel derivative
    float gy = ...        // compute vertical sobel derivative
}
```

Block: shares memory



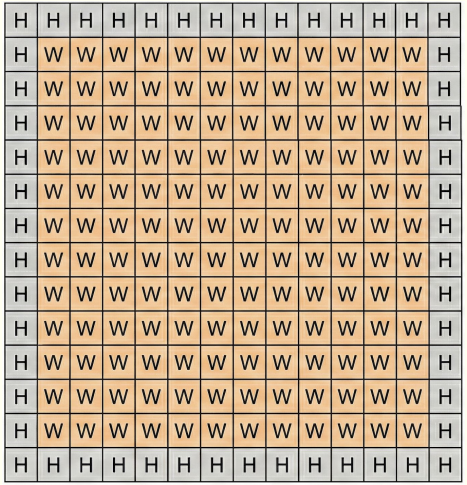
H = Halo (load data only)

W = Work threads (load + compute internal pixels)

H-S with Shared Memory


```
__global__ void horn_schunck_iteration(...){

  // Step 1: Shared mem. declaration for 5 arrays
  __shared__ float s_U[18][18], s_V[18][18], s_Ix[18][18], ...

  int x = bx * 16 + tx - 1;   // find global x coordinate
  int y = by * 16 + ty - 1;   // find global y coordinate

  // Step 2: Load 5 arrays into shared memory
  s_U[ty][tx] = U_old[idx];
  s_V[ty][tx] = V_old[idx];    ...
  __syncthreads();       // barrier

  // Step 3: only compute on internal pixels
  if (tx >= 1 && tx <= 16 && ty >= 1 && ty <= 16) {
    U_avg = (1/12)*corners + (1/6)*sides;    // 8-neighbor avg
    V_avg = ...
    U_new[idx] = ...;      V_new[idx] = ...
}
```

Block: shares memory



H = Halo (load data only)

W = Work threads (load + compute internal pixels)

H-S with Distributed Memory

Frame are distributed across MPI ranks:

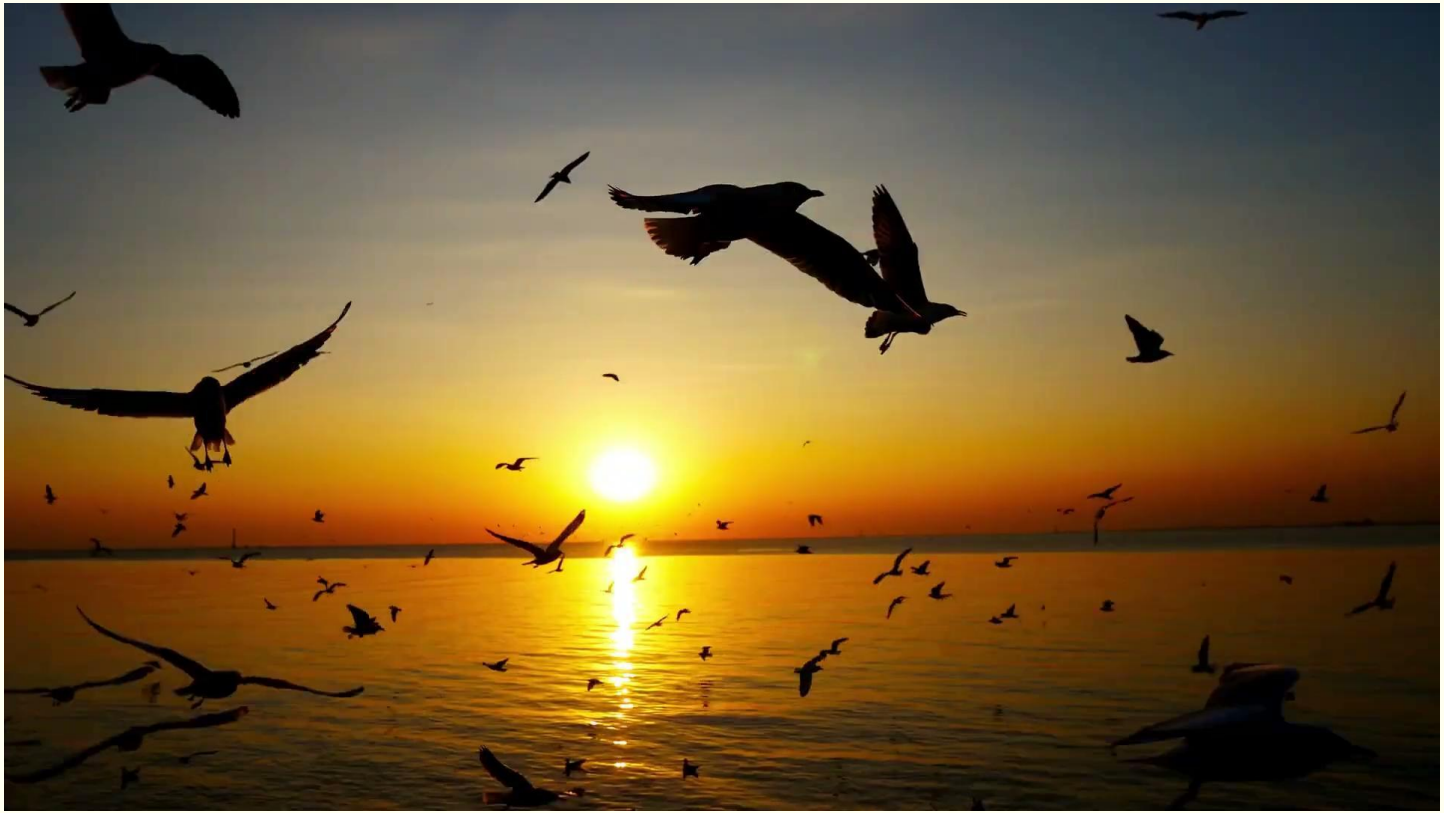
- Rank 0: frames 0-16
- Rank 1: frames 16-32
- Rank 2: frames 32-48
- Rank 3: frames 48-63

Flow magnitudes need to be normalized, so the ranks:

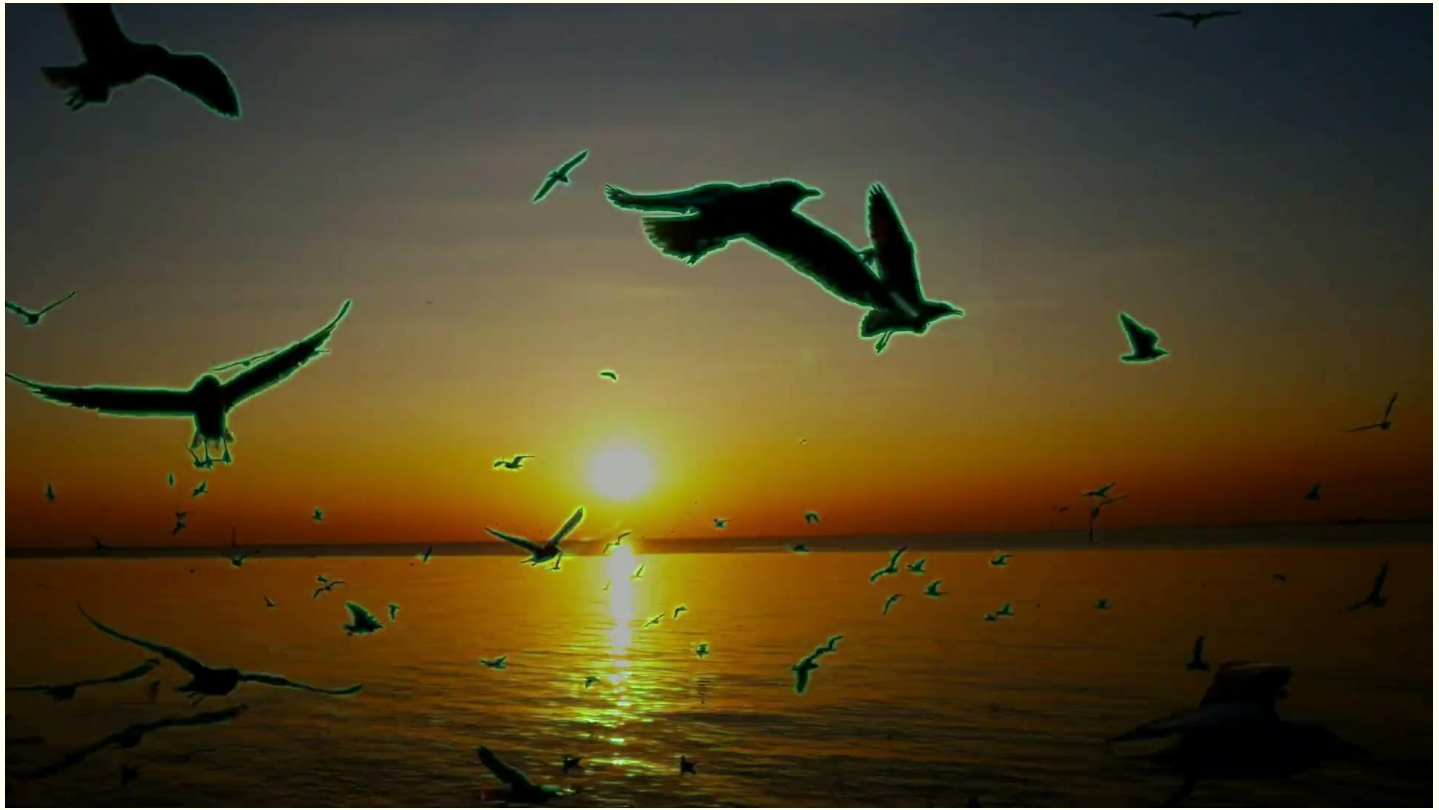
- Share local max
- Receive global max

```
// Rank 0 loads video
if (rank == 0)
    load_frames_and_metadata();

// Rank 0 distributes frame subsets to each rank
MPI_Scatterv(flat.data(), send_counts, displs, ...,
recv_buf.data(), ...);

// Each rank runs GPU optical flow on its local frames
local_max_mag = process_frame_range(local_frames, ...);

// All ranks share local max → all receive global max
MPI_Allreduce(&local_max_mag, &global_max_mag, 1,
MPI_FLOAT, MPI_MAX, MPI_COMM_WORLD);

// All ranks send flow results → Rank 0 collects
MPI_Gatherv(local_U.data(), ..., U_all.data(), ...);
```

Original Video



Flow Field Result (blended)



Flow Field Result (overlay only)

Roofline Analysis on HS

$$u^{k+1} = \overline{u}^k - \frac{I_x(I_x\overline{u}^k + I_y\overline{v}^k + I_t)}{\alpha^2 + I_x^2 + I_y^2}$$
$$v^{k+1} = \overline{v}^k - \frac{I_y(I_x\overline{u}^k + I_y\overline{v}^k + I_t)}{\alpha^2 + I_x^2 + I_y^2}$$

44 FLOPS / pixel:

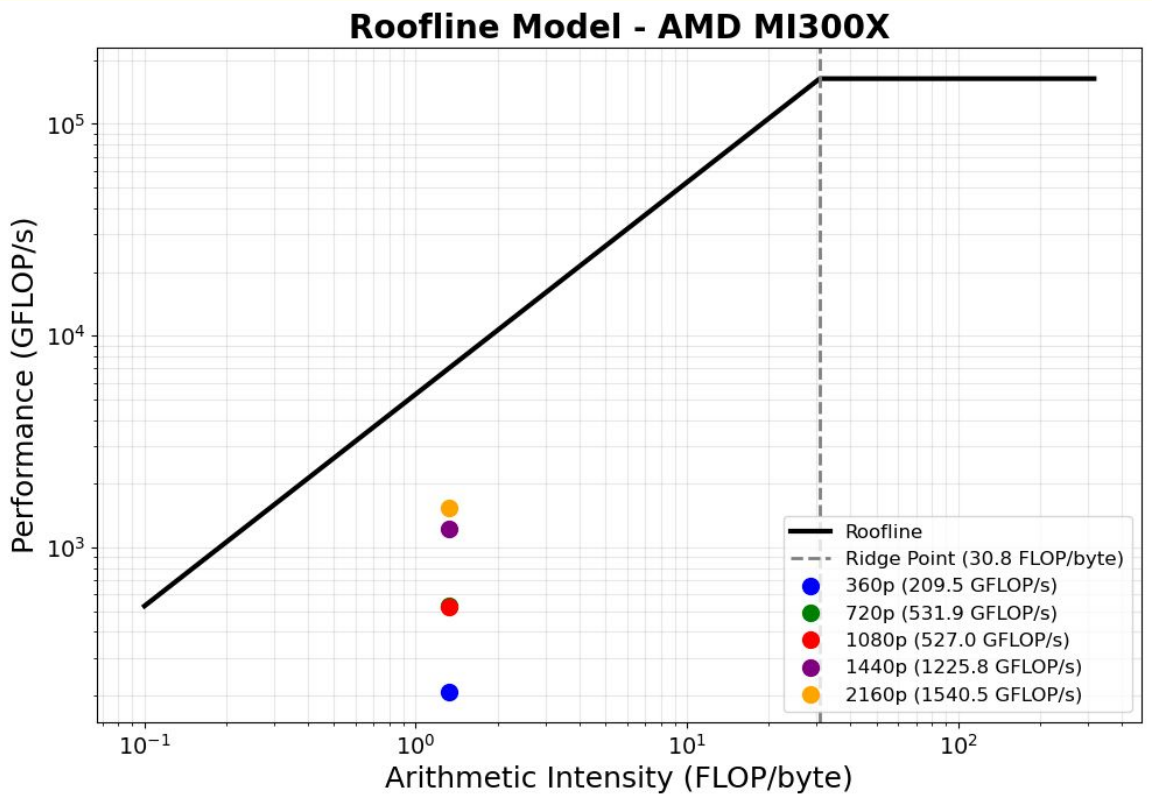
8-point average for U: 8 mults + 7 adds = **15** FLOPs. Same for V: **15** FLOPs.
Numerator: 2 mults + 2 adds = **4** FLOPs. Denominator: 2 + 2 = **4** FLOPs.
Update U and V: 2 × (1 mult + 1 div + 1 sub) = **6** FLOPs.

33.3 bytes / pixel:

Block loads: 5 arrays × 18×18 tiles = 6,480 bytes.
Block writes: 2 arrays × 16×16 tiles = 2,048 bytes. Total: 8,528 bytes per block
Per pixel: 8,528 bytes ÷ 256 pixels = 33.3 bytes/pixel

$$AI = \frac{44\text{FLOPs}}{33.3\text{bytes}} = 1.32\text{FLOP/byte}$$

Roofline Analysis on HS



The H-S kernel is memory-bound.

Limitations

- Load balancing
  - When we dealt with longer or higher resolution videos, we ran into data overflow issues
  - If we streamed our frames in batches and only sent new work to a rank after it finished its current batch, we could keep memory usage bounded and improve load balancing for very long / 4K videos.
- Video writing
  - To increase parallelization and decrease processing time further, we could offload the video writing step to the GPU.
  - Using a GPU-accelerated encoder would let us write frames faster and avoid the bottleneck of finishing the main computation.

Thank you